Lab 7: Applications of Linked Stacks and Queues

Learning Objectives

- Apply a linked stack to convert a simple infix expression into postfix notation.
- Use a linked queue to simulate customers being served automatically in first-in, first-out order.

Starter Files

Download both starter files from Canvas. Linked stack and linked queue functions are provided. Complete only the TODO sections; do not use STL stack or queue.

- Lab7_Activity1.cpp
- Lab7_Activity2.cpp

Activity 1 : Infix-to-Postfix Conversion

Scenario: Convert an infix expression into postfix notation using the linked stack provided in the starter file.

- Complete precedence(char op): * and / have higher precedence than + and -.
- Complete infixToPostfix(string infix) using push(), pop(), peek(), and isEmpty().
- Treat each letter or digit as one operand. Append operands directly to the output.
- Handle parentheses and equal-precedence operators as covered in lecture.
- Test the expressions below and compare them with the expected output.

Scope: Valid expressions with single-character operands, +, -, *, /, and balanced parentheses; no spaces, multi-digit operands, exponentiation, or unary minus.

Infix expression	Expected postfix
A+B	AB+
A+B*C	ABC*+
A*B+C	AB*C+
(A+B)*C	AB+C*
A+B*C-D	ABC*+D-
A*(B+C)/D	ABC+*D/

C++ Quick Reference : Activity 1

The starter file includes <cctype> for isalnum(). These operations may help:

Operation	**Example**	**Purpose**
Read characters	for (char ch : infix)	Visit each character
Check operand	isalnum(ch)	Identify letters/digits
Append character	postfix += ch;	Build output
Compare symbol	ch == '('	Identify parentheses

Stack methods	push(), pop(), peek(), isEmpty()	Work with operators

Activity 2 : Automatic Customer Service Queue

Scenario: Five customers are waiting for service. Each customer takes 10 seconds to serve. After service, the customer leaves the front of the linked queue and the next customer is served automatically (FIFO).

Complete the TODO sections inside main():

1. Add these five customers in order using enqueue(): Ali, John, Sara, David, Emma.
2. Display the initial waiting queue using displayQueue().
3. Use a loop that continues while the queue is not empty.
4. For each customer, show who is being served, wait for 10 seconds, then remove that customer using dequeue().
5. Print a service-completed message; repeat automatically until all five customers are served.
6. Display a final message and verify that the queue is empty.

Helpful Hints : Activity 2

Include the following libraries to pause program execution:

```
#include <thread>
#include <chrono>
```

Use this statement to simulate 10 seconds of service:

```
this_thread::sleep_for(chrono::seconds(5));
```

For quicker testing, you may temporarily change 5 to 1 second; restore 5 seconds before submitting. Five customers require about 25 seconds of total simulated service time.

Example output (shortened):

```
Waiting customers: Nancy John Sarah David Emma
Now serving: Nancy
Service completed: Nancy (5 seconds)
Now serving: John
Service completed: John (10 seconds)
…
…
All customers have been served.
Queue is empty.
```